YNOV CAMPUS / JOUR 5

Jour 5 Mesurer et améliorer

Plans, indexation, statistiques et partitionnement. Apprenez avec des exemples commentés, puis entraînez-vous dans TP05.

PostgreSQL 18 | 2e année de bachelor | Édition pédagogique progressive

Partir d’une question qui compte pour les utilisateurs

CampusMarket doit afficher rapidement les commandes récentes d’un étudiant pendant que d’autres étudiants passent commande. La performance désigne les ressources et le temps nécessaires pour produire un résultat correct. Précisez d’abord la question métier, les lignes attendues et le temps de réponse acceptable. Une requête qui renvoie les mauvaises commandes ne constitue jamais une amélioration réussie. Aujourd’hui, nous étudions work.bench_orders, une table synthétique jetable de 300,000 commandes, au lieu d’utiliser le jeu de données pédagogique de dix lignes comme test de vitesse.

La performance désigne le temps et les ressources nécessaires pour produire un résultat correct.

Suivez le raisonnement

1. Précisez les lignes et colonnes dont l’écran a réellement besoin.
2. Notez la requête actuelle et son temps de réponse avant de la modifier.
3. Comparez le même résultat après un seul changement justifié.

Préparer le banc d’essai avant de mesurer les requêtes

Un banc d’essai est un jeu de données contrôlé servant à comparer des façons d’effectuer le même travail. Exécutez le script fourni ci-dessous avant les exemples de ce jour. Il crée work.bench_orders avec 300,000 commandes synthétiques. Ces identifiants d’étudiants synthétiques sont indépendants de campus.students. La table est volontairement plus grande que notre jeu de données pédagogique de dix commandes, car les petites tables ne révèlent pas de différences utiles entre les méthodes d’accès.

TERMINAL / COMMANDES CLIENT

```
docker compose exec -T db psql -U course_admin \
  -d campusmarket -f /course/03_benchmark.sql
```

Résultat / données d’exemple

Colonne	Signification
order_id	ID de commande synthétique
student_id	Identifiant de client synthétique
ordered_at	Instant de la commande
status	paid ou pending
campus_id	Code de campus synthétique
total	Montant de commande synthétique

Le planificateur prévoit le nombre de lignes correspondantes

Le planificateur de requêtes est le composant de PostgreSQL qui choisit comment exécuter SQL. Un plan d’exécution est son arbre d’opérations, comme la lecture des lignes, les jointures et le tri. Les statistiques sont des résumés stockés du contenu des tables, utilisés pour les prévisions. La cardinalité désigne un nombre de lignes ; la sélectivité, la proportion de lignes satisfaisant une condition. Dans ce banc d’essai, student_id = 42 correspond à 30 lignes sur 300,000 ; sa sélectivité vaut donc 0.0001, soit 0.01%.

Résultat / données d’exemple

Condition	Cardinalité exacte	Sélectivité
Toutes les commandes du banc d’essai	300,000	100%
student_id = 42	30	0.01%
status = 'pending'	30,000	10%
status = 'paid'	270,000	90%

EXPLAIN montre un parcours proposé sans exécuter SELECT

Placez EXPLAIN avant une requête complète pour afficher le plan d’exécution proposé. EXPLAIN seul planifie ce SELECT sans récupérer ses lignes de résultat. Notre question porte sur les trois premiers identifiants de commandes de l’étudiant 42. ORDER BY explicite l’ordre demandé et LIMIT plafonne les lignes renvoyées. Le planificateur peut choisir parmi les parcours disponibles ; un plan renseigne sur ce choix, sans garantir que le même parcours sera toujours utilisé.

SQL / lire l’instruction une clause à la fois

```sql
EXPLAIN
SELECT order_id
FROM work.bench_orders
WHERE student_id = 42
ORDER BY order_id
LIMIT 3;
```

Sens des mots-clés

EXPLAIN — Afficher le plan d’exécution de l’instruction suivante.

ORDER BY — Demander un ordre défini pour les lignes renvoyées.

LIMIT 3 — Renvoyer au maximum trois lignes.

Lire ce plan abrégé de cinq lignes en partant du bas

Ce plan de cinq lignes est illustratif et abrégé ; ses nombres sont pédagogiques et ne proviennent pas d’un relevé du banc d’essai. Seq Scan signifie examiner les lignes de la table, puis Filter conserve l’étudiant 42. Sort ordonne ces correspondances et Limit en renvoie trois. Les valeurs rows prévoient les lignes émises par une opération, pas toutes celles qu’elle examine. Le coût est un score de comparaison du planificateur : les deux nombres représentent le coût initial et le coût total. Un coût de 8150 ne signifie pas 8150 millisecondes.

Résultat / données d’exemple

Ligne de plan illustrative	Interprétation simple
Limit (cost=8150.00..8150.01 rows=3)	Émettre au maximum 3 lignes.
→ Sort (cost=8150.00..8150.08 rows=30)	Trier environ 30 correspondances prévues.
Sort Key: order_id	Utiliser order_id croissant.
→ Seq Scan on bench_orders (rows=30)	Parcourir les lignes de la table ; prévoir 30 sorties.
Filter: (student_id = 42)	Écarter les lignes appartenant aux autres.

ANALYZE mesure l’exécution ; BUFFERS montre l’activité des pages

Un tampon est un emplacement mémoire contenant une page de base de données, bloc de taille fixe comprenant des lignes ou des entrées d’index. Les entrées/sorties, abrégées I/O, désignent les transferts de données entre le stockage et la mémoire. EXPLAIN ANALYZE exécute réellement l’instruction et ajoute les nombres de lignes et les temps mesurés. BUFFERS indique l’activité des pages : shared hit trouve une page déjà en mémoire PostgreSQL, tandis que shared read la charge dans cette mémoire, éventuellement depuis le cache du système d’exploitation.

SQL / lire l’instruction une clause à la fois

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id
FROM work.bench_orders
WHERE student_id = 42
ORDER BY order_id
LIMIT 3;
```

Sens des mots-clés

Option ANALYZE — Exécuter cette instruction et rapporter les détails d’exécution mesurés.

BUFFERS — Inclure l’activité des pages de base de données dans le plan.

shared hit — Une page consultée était déjà en mémoire partagée PostgreSQL.

shared read — Une page a été chargée en mémoire partagée PostgreSQL.

Comparer les estimations aux lignes réelles pour la même opération

Ces mesures illustratives décrivent l’opération de parcours avant la limite finale de trois lignes. Le planificateur prévoyait 300 lignes correspondantes, mais l’exécution en a trouvé 30, soit une surestimation d’un facteur dix. Actual rows est le nombre de sorties mesuré ; loops indique le nombre d’exécutions de l’opération. Si loops dépasse un, actual rows et les temps sont des moyennes par boucle. Les nombres de pages révèlent un parcours important, même si seuls trois identifiants de commandes atteignent finalement l’écran.

Résultat / données d’exemple

Mesure illustrative du parcours	Valeur	Signification
Estimation rows	300	Sortie prévue pour ce parcours.
actual rows / loops	30 / 1	30 sorties en une exécution.
Rows Removed by Filter	299,970	Lignes parcourues écartées ici.
Buffers: shared hit / read	2,500 / 400	Accès aux pages, pas lignes de résultat.
Execution Time	24.0 ms	Exécution complète mesurée dans cet exemple.

À vous de jouer : expliquer un écart sans deviner une solution

Un rapport d’assistance sélectionne les commandes pending du banc d’essai. Au niveau du parcours, le plan estime 3,000 lignes en sortie, mais EXPLAIN ANALYZE indique 30,000 lignes réelles avec loops = 1. Exprimez l’erreur d’estimation sous forme de rapport, calculez la proportion exacte de correspondances parmi les 300,000 commandes et déterminez si cette seule observation prouve qu’un index sera utile. Indice : distinguez la précision de la prévision du travail physique du parcours choisi.

Suivez le raisonnement

1. Calculez le nombre de lignes réelles divisé par le nombre de lignes estimées pour ce parcours.
2. Divisez les lignes correspondantes par le total des lignes pour obtenir la sélectivité.
3. Nommez une mesure supplémentaire nécessaire avant de choisir un changement.

Réponse : dix fois plus de lignes ne désigne pas un remède unique

Le parcours a émis dix fois le nombre de lignes estimé : 30,000 divisé par 3,000 vaut 10. Sa sélectivité réelle est de 30,000 divisé par 300,000, soit 10%. Le planificateur a sous-estimé les correspondances, ce qui peut influencer les choix ultérieurs de jointure ou de tri. Cela n’établit pas qu’un index est plus rapide. Actualiser les statistiques et comparer l’activité réelle des pages et le temps écoulé sont des pistes raisonnables, tout en préservant exactement le résultat demandé.

Résultat / données d’exemple

Question	Corrigé expliqué
Facteur de sous-estimation	30,000 ÷ 3,000 = 10
Proportion de correspondances	30,000 ÷ 300,000 = 10%
Éléments suivants	Fraîcheur des statistiques, tampons, chronométrages répétés

Lire ensemble les estimations, les lignes réelles et le travail mesuré.

Un B-tree conserve un parcours trié distinct vers les lignes de la table

Un index de base de données est une structure physique stockée séparément de la table, avec des valeurs de recherche et des références aux lignes de la table. Un B-tree conserve ces valeurs dans des pages triées reliées par des pages supérieures qui guident la recherche. Pour l’étudiant 42, PostgreSQL peut naviguer vers 42 et examiner les entrées correspondantes au lieu de vérifier la ligne de chaque étudiant. L’index occupe du stockage et doit être entretenu lorsque les lignes concernées changent ; il ne réécrit pas la table dans cet ordre.

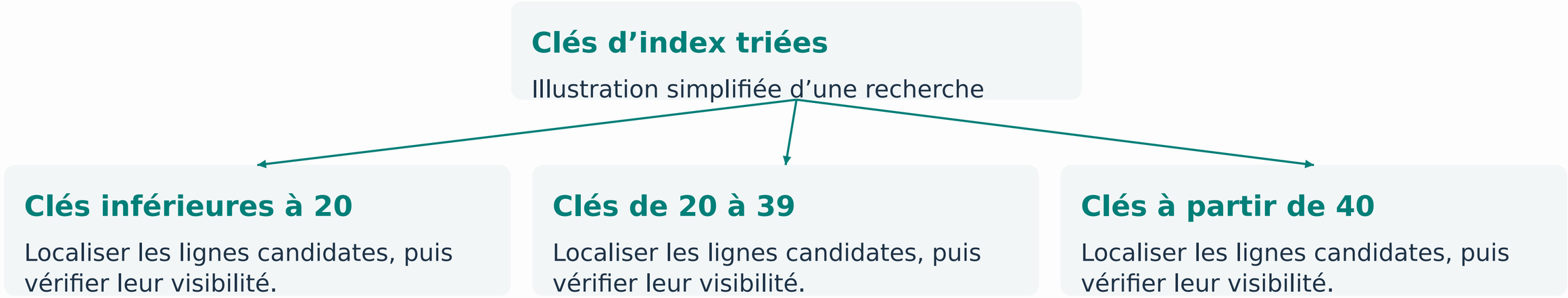
Un B-tree est un index qui parcourt des clés de recherche triées à travers des niveaux de pages reliés.

Résultat / données d’exemple

Clé d’index triée	Références dans cet extrait conceptuel	Recherche de 42
41	commande 40 ; commande 10040	Avant le groupe cible.
42	commande 41 ; commande 10041	Parcourir les entrées correspondantes.
43	commande 42 ; commande 10042	S’arrêter après le groupe cible.

Voir l’index comme un parcours vers des lignes candidates

Un index est une structure supplémentaire qui aide PostgreSQL à localiser des lignes candidates dans la table. Ce schéma simplifié de clés triées illustre le choix d’une plage plus petite au lieu de lire toutes les valeurs. Un véritable B-tree comporte des pages de base de données et de nombreuses entrées. Trouver une clé ne prouve pas automatiquement que sa version dans la table est visible pour la transaction actuelle.



Construire un index candidat pour une question récurrente

CREATE INDEX construit une structure de recherche supplémentaire ; ON désigne sa table et la colonne entre parenthèses constitue la clé de recherche. PostgreSQL utilise ici un B-tree par défaut. Nous créons un index candidat sur student_id, puis répétons la même requête d'historique des commandes. Cet index peut aider à localiser les 30 lignes correspondantes, mais un tri distinct peut rester nécessaire, car order_id n'est pas sa clé. Mesurez le plan au lieu de supposer que la création d'un index force PostgreSQL à l'utiliser.

SQL / lire l'instruction une clause à la fois

```sql
CREATE INDEX bench_student_idx
ON work.bench_orders (student_id);

SELECT order_id, total
FROM work.bench_orders
WHERE student_id = 42
ORDER BY order_id
LIMIT 3;
```

Sens des mots-clés

CREATE INDEX — Construire un index physique nommé.

ON — Préciser la table dont l'index référence les lignes.

(student_id) — Utiliser les valeurs de student_id comme clé de recherche.

L’index change le parcours, tandis que ces lignes restent identiques

Les trois premiers identifiants correspondants sont 41, 10041 et 20041. Chacun a un total de 41.00, car le générateur du banc d’essai répète les montants toutes les 10,000 commandes. Ce sont des résultats de données exacts, contrairement aux nombres illustratifs des plans. Une expérience d’indexation réussie préserve ces lignes et leur ordre tout en réduisant le travail mesuré pour la charge visée. PostgreSQL peut choisir un Index Scan, un parcours bitmap regroupant les accès à la table par page, ou un autre parcours si ses estimations le favorisent.

Résultat / données d’exemple

order_id	total
41	41.00
10041	41.00
20041	41.00

Un index n’est utile que si son bénéfice pour la charge justifie ses coûts de stockage et d’écriture.

Faire une comparaison équitable avant et après

Un banc d’essai est une mesure contrôlée servant à comparer des solutions. Gardez comparables les données, le résultat SQL, les paramètres, les réglages de session et la charge concurrente. Effectuez une exécution de préchauffage, puis plusieurs exécutions mesurées avant et après l’unique changement. Un cache chaud signifie que les pages utiles sont déjà en mémoire. Comparez les exécutions à chaud entre elles et résumez les temps répétés par la médiane, valeur centrale après tri, afin qu’une exécution inhabituelle ne domine pas.

Suivez le raisonnement

1. Relevez le résultat de référence, le plan et cinq exécutions mesurées comparables.
2. Modifiez un seul index candidat ou une condition de requête et confirmez que les résultats sont identiques.
3. Préchauffez la nouvelle version, répétez les mesures et comparez les médianes et l’activité des pages.

Une médiane mesurée plus basse est un indice, pas une garantie de vitesse

Ces temps sont des données pédagogiques illustratives, pas des mesures du kit fourni. En triant les cinq temps de référence, 25 millisecondes devient la valeur centrale ; celle du candidat est de 4 millisecondes. Cela justifie de tester davantage le candidat pour cette question et dans ces conditions. Cela ne promet pas le même rapport sur une autre machine, une table plus grande ou pendant de nombreuses écritures concurrentes. Continuez à vérifier les lignes correctes, l’activité des pages, le stockage et le coût d’entretien d’un index supplémentaire.

Résultat / données d’exemple

Version	Cinq exécutions à chaud, ms	Médiane, ms
Référence	24, 25, 25, 26, 80	25
Un index candidat	3, 4, 4, 5, 9	4

Indiquez les conditions de mesure et l’incertitude avec toute amélioration.

Placer l’égalité avant la plage dans cette clé multicolonne

Un index multicolonne possède plusieurs colonnes de clé de recherche, ordonnées de gauche à droite. Pour les commandes ultérieures d’un étudiant, student_id = 42 fixe la première clé, puis order_id >= 20000 sélectionne une plage dans le groupe de cet étudiant. Ce B-tree peut aussi fournir l’ordre order_id pour l’étudiant fixé. La conception suit la question réelle ; inverser les colonnes crée une structure triée différente et peut imposer de chercher dans une zone bien plus large.

SQL / lire l’instruction une clause à la fois

```sql
CREATE INDEX bench_student_order_idx
ON work.bench_orders (student_id, order_id);

SELECT order_id, total
FROM work.bench_orders
WHERE student_id = 42
  AND order_id >= 20000
ORDER BY order_id
LIMIT 3;
```

Sens des mots-clés

(student_id, order_id) — Trier les clés d’index par étudiant, puis par commande de cet étudiant.

= 42 — Fixer la première clé de recherche à une valeur.

>= 20000 — Sélectionner une plage de la clé de recherche suivante.

La clé triée rend cette plage visible

L’extrait affiché est conceptuel ; les identifiants renvoyés 20041, 30041 et 40041 sont exacts. L’égalité sur 42 atteint le groupe d’un étudiant, puis la plage commence à 20000. Une condition sur une clé ultérieure seule peut parfois utiliser le skip scan de PostgreSQL 18, qui effectue des recherches répétées parmi les valeurs de tête. Le skip scan est particulièrement intéressant lorsque la première clé omise a peu de valeurs distinctes. Il ne rend pas l’ordre des colonnes indifférent et ne garantit pas le choix d’un index.

Résultat / données d’exemple

student_id	order_id	Effet de la requête
42	10041	Même étudiant ; avant la plage.
42	20041	Première ligne renvoyée ; total 41.00.
42	30041	Deuxième ligne renvoyée ; total 41.00.
42	40041	Troisième ligne renvoyée ; total 41.00.
43	20042	Première clé différente ; exclue.

À vous de jouer : choisir une clé pour un écran concret

L’écran d’assistance recherche toujours un campus, puis demande les commandes à partir d’un identifiant donné, renvoyées par ordre d’identifiant. Comparez les clés candidates (campus_id, order_id) et (order_id, campus_id). Choisissez le candidat de départ naturel et expliquez les rôles de l’égalité, de la plage et de l’ordre. Expliquez ensuite pourquoi on ne doit pas déclarer qu’une requête portant uniquement sur order_id ne peut pas utiliser le premier index. Indice : distinguez une règle de conception utile d’une interdiction absolue.

Suivez le raisonnement

1. Placez la colonne fixée par égalité avant la plage d’identifiants demandée.
2. Expliquez comment cet ordre peut aussi servir ORDER BY.
3. Mentionnez le skip scan de PostgreSQL 18 sans promettre qu’il sera choisi.

Réponse : suivre l’écran, puis examiner le plan choisi

Le candidat naturel est (campus_id, order_id) : l’égalité sélectionne un groupe de campus et la plage d’identifiants y est contiguë. Dans ce campus fixé, l’ordre de l’index correspond aussi à ORDER BY order_id. Une requête sur les seules commandes peut encore utiliser cet index dans certaines circonstances, notamment avec un skip scan de PostgreSQL 18 sur les deux valeurs de campus. L’index de clé primaire existant sur order_id est un autre candidat ; le plan réel et les mesures déterminent donc le parcours intéressant.

Résultat / données d’exemple

Élément de charge	Rôle dans le candidat
campus_id = 2	Fixer le groupe de tête.
order_id >= 20000	Commencer une plage dans ce groupe.
ORDER BY order_id	Lire ce groupe dans l’ordre existant de sa clé.

Concevoir pour les conditions et l’ordre réels, puis vérifier le choix du planificateur.

Exprimer un jour comme une plage d’horodatages semi-ouverte

Une condition est sargable lorsqu’un index peut l’utiliser pour restreindre directement la recherche. Appliquer une fonction ou une conversion à chaque valeur d’une colonne indexée peut empêcher un index ordinaire sur cette colonne de le faire. La première requête convertit chaque horodatage en date. La seconde décrit le même jour UTC par une plage semi-ouverte : inclure son début et exclure le début suivant. Cela conserve ordered_at lui-même comme clé de recherche et évite de deviner le dernier instant du jour.

SQL / lire l’instruction une clause à la fois

```sql
SET TIME ZONE 'UTC';
SELECT count(*) FROM work.bench_orders
WHERE CAST(ordered_at AS date) = DATE '2025-01-02';

SELECT count(*) FROM work.bench_orders
WHERE ordered_at >= TIMESTAMPTZ '2025-01-02 00:00+00'
  AND ordered_at < TIMESTAMPTZ '2025-01-03 00:00+00';
```

Sens des mots-clés

Condition sargable — Condition permettant de restreindre directement une recherche d’index.

Plage semi-ouverte — Inclure la borne inférieure et exclure la borne supérieure.

SET TIME ZONE — Définir le fuseau horaire de session utilisé pour interpréter les horodatages et dates.

TIMESTAMPTZ — Écrire une valeur d’horodatage avec un décalage de fuseau horaire.

Indexer une valeur calculée lorsque la question l’exige

Un index d’expression stocke une valeur de recherche calculée plutôt que la seule colonne d’origine. LOWER convertit les lettres en minuscules. Si la recherche d’adresses e-mail sans distinction de casse est récurrente, indexer lower(email) fournit un parcours pour la même expression dans WHERE. La requête compare une valeur de colonne normalisée à une valeur de recherche normalisée. Sur le jeu de données de six étudiants, PostgreSQL peut encore préférer un parcours séquentiel, car lire une table aussi petite coûte peu ; cette diapositive montre la correspondance des expressions, sans affirmer un gain de vitesse.

SQL / lire l’instruction une clause à la fois

```sql
CREATE INDEX students_email_lower_idx
ON campus.students (lower(email));

SELECT student_id, email
FROM campus.students
WHERE lower(email) = lower('ALICE@EXAMPLE.TEST');
```

Sens des mots-clés

Index d’expression — Index dont la clé est le résultat d’un calcul.

lower(text) — Renvoyer le texte avec les lettres converties en minuscules.

(lower(email)) — Stocker la même valeur calculée que celle recherchée par la requête.

La normalisation modifie la comparaison, pas l’adresse e-mail stockée

La recherche renvoie l’écriture existante de l’adresse d’Alice, alice@example.test, et l’identifiant d’étudiant 1. LOWER a rendu la comparaison insensible aux majuscules fournies. Il n’a pas mis à jour la ligne stockée. Un index simple sur email et un index sur lower(email) stockent des clés de recherche différentes ; les comparaisons qu’ils prennent en charge diffèrent donc. Choisissez un index d’expression pour une question récurrente stable et rappelez-vous que stocker des clés calculées ajoute du travail d’écriture et de stockage, même lorsque l’optimiseur choisit un parcours de table.

Résultat / données d’exemple

student_id	email
1	alice@example.test

L’expression de la requête doit correspondre au type de valeur stocké pour la recherche.

Un index partiel ne stocke que les lignes satisfaisant sa condition

Un prédicat est une condition évaluée à vrai ou faux, éventuellement inconnu en présence de NULL. Un index partiel ne contient que les lignes pour lesquelles son prédicat est vrai. Ici, la file des commandes pending a besoin d’un parcours ordonné des commandes pending ; l’index stocke donc order_id uniquement pour ce sous-ensemble. Sa clause WHERE définit l’appartenance à l’index ; la clause WHERE du SELECT définit toujours la réponse et doit permettre à PostgreSQL de prouver que l’index contient toutes les correspondances possibles.

SQL / lire l’instruction une clause à la fois

```sql
CREATE INDEX bench_pending_idx
ON work.bench_orders (order_id)
WHERE status = 'pending';

SELECT order_id, status
FROM work.bench_orders
WHERE status = 'pending'
  AND order_id >= 299950
ORDER BY order_id
LIMIT 3;
```

Un index partiel ne stocke que les lignes satisfaisant sa condition / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

Index partiel — Index ne contenant que les lignes satisfaisant sa condition.

Prédicat — Condition ; seules les lignes vraies sont admissibles ici.

WHERE dans CREATE INDEX — Choisir les lignes de table qui reçoivent des entrées d’index.

La file pending utilise un ensemble admissible plus petit

La file renvoie les commandes pending 299950, 299960 et 299970. L’index partiel du banc d’essai peut contenir 30,000 lignes pending plutôt que les 300,000 lignes, mais sa taille physique doit être mesurée. Une requête sur les commandes paid ne peut pas utiliser cet index comme seul parcours pour trouver les lignes paid, car elles en sont absentes. La requête doit établir la condition pending ; une condition plus large autorisant à la fois les lignes paid et pending ne peut pas s’appuyer sur ce seul index en toute sécurité.

Résultat / données d’exemple

order_id	status
299950	pending
299960	pending
299970	pending

INCLUDE transporte des données de sortie sans en faire une clé de recherche

Un index couvrant contient toutes les valeurs de colonnes nécessaires à une requête. INCLUDE stocke des colonnes de sortie supplémentaires dans un index sans les ajouter à sa clé de recherche ordonnée. Ici, student_id et order_id guident la recherche, tandis que total est transporté pour que le résultat puisse être obtenu depuis l'index. Un parcours d'index seul peut renvoyer les valeurs nécessaires depuis les entrées d'index ; il lui faut toujours un moyen de vérifier que ces lignes sont visibles pour la transaction.

SQL / lire l'instruction une clause à la fois

```sql
CREATE INDEX bench_cover_idx
ON work.bench_orders (student_id, order_id)
INCLUDE (total);

EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, total
FROM work.bench_orders
WHERE student_id = 42
ORDER BY order_id
LIMIT 3;
```

Sens des mots-clés

INCLUDE — Stocker des colonnes renvoyées supplémentaires sans les utiliser comme clés ordonnées.

Index couvrant — Index contenant toutes les valeurs de colonnes nécessaires à une requête donnée.

Parcours d'index seul — Lire les valeurs nécessaires dans les entrées d'index, avec vérifications de visibilité.

La visibilité peut encore nécessiter des accès à la table

Le heap est le stockage principal des tables de PostgreSQL. La carte de visibilité indique si toutes les lignes d’une page du heap sont connues comme visibles par toutes les transactions. Lors d’un parcours d’index seul, une marque de visibilité globale peut éviter de consulter cette page ; sans elle, PostgreSQL vérifie le heap. Ces détails de plan sont des possibilités illustratives, pas des promesses. Les deux renvoient les mêmes trois identifiants de commandes et le total 41.00, mais Heap Fetches révèle si le parcours supposé d’index seul a encore consulté les lignes de la table.

Résultat / données d’exemple

Détail de plan illustratif	Table peu modifiée et entretenue	Pages récemment modifiées
Opération	Index Only Scan	Index Only Scan
Lignes réelles en sortie	3	3
Heap Fetches	0	3
Identifiants du résultat	41, 10041, 20041	41, 10041, 20041

Reconnaître GIN et BRIN avant de les choisir

B-tree n’est pas la seule structure d’index. GIN, Generalized Inverted Index, associe des valeurs composantes aux lignes qui les contiennent, ce qui le rend utile pour les recherches d’appartenance prises en charge, comme des mots dans des documents ou des éléments dans des collections. BRIN, Block Range Index, stocke des résumés de plages de pages de table, ce qui le rend utile lorsque les valeurs suivent l’ordre physique du stockage. Aucun ne signifie automatiquement plus rapide. Choisissez la structure adaptée à la recherche et à l’organisation des données, puis examinez les comparaisons qu’elle prend en charge avant de l’implémenter.

Résultat / données d’exemple

Structure	Exemple simple de recherche	Raison possible de son adéquation
B-tree	student_id = 42 ; une plage de order_id	Parcourir des clés individuelles triées.
GIN	Trouver les documents contenant keyboard	Trouver les lignes par leurs valeurs composantes indexées.
BRIN	Trouver des dates dans un très grand journal chronologique	Ignorer les plages de pages dont les résumés ne peuvent pas correspondre.

Un même résultat de jointure peut avoir différents parcours physiques

Un algorithme de jointure est la procédure utilisée par PostgreSQL pour faire correspondre les lignes de deux entrées. SQL décrit quelles lignes d'étudiants et de commandes vont ensemble ; il n'impose généralement pas cette procédure. Cette requête demande les trois premières commandes du campus avec les noms de leurs étudiants. Nous connaissons déjà la jointure logique du Jour 2. L'étude des performances demande maintenant combien de lignes fournit chaque entrée, si des index ou un ordre utiles existent et comment le planificateur choisit une procédure de correspondance.

SQL / lire l'instruction une clause à la fois

```sql
SELECT o.order_id, s.full_name
FROM campus.orders AS o
JOIN campus.students AS s
  ON s.student_id = o.student_id
WHERE o.order_id <= 103
ORDER BY o.order_id;
```

Sens des mots-clés

Algorithme de jointure — Procédure physique trouvant les lignes d'entrée correspondantes.

ON s.student_id = o.student_id — Associer chaque commande à son étudiant par égalité.

Appliquer trois procédures de correspondance aux mêmes lignes

Nos trois lignes de commandes portent les identifiants d’étudiants 1, 2 et 1. Une boucle imbriquée prend chaque commande et recherche de nouveau dans l’entrée des étudiants, éventuellement via un index. Une jointure par hachage construit pour une entrée une structure de recherche appelée table de hachage, puis la sonde avec les clés de l’autre entrée. Une jointure par fusion parcourt deux entrées ordonnées par la clé de jointure en alignant les clés égales. Chaque méthode doit préserver les correspondances multiples ; Alice apparaît donc pour les commandes 101 et 103.

Résultat / données d’exemple

Procédure	Actions concrètes sur ces trois commandes
Boucle imbriquée	101 → rechercher l’étudiant 1 ; 102 → rechercher 2 ; 103 → rechercher encore 1.
Jointure par hachage	Stocker 1 → Alice et 2 → Bilal ; sonder avec les clés 1, 2, 1.
Jointure par fusion	Lire les clés de commandes triées (1:101, 1:103, 2:102) à côté des étudiants (1:Alice, 2:Bilal).

La réponse de la jointure est indépendante de la procédure de correspondance

Le résultat exact contient les commandes 101 et 103 pour Alice et la commande 102 pour Bilal, affichées dans l’ordre order_id. Changer l’algorithme physique de jointure ne doit pas changer ces correspondances logiques. L’ordre interne par clé d’étudiant d’une jointure par fusion ne supprime pas la nécessité du ORDER BY order_id demandé. Lorsqu’une jointure est lente, examinez la taille des deux entrées et les erreurs d’estimation avant d’essayer de changer d’algorithme ; un filtre antérieur ou une multiplication erronée des correspondances peut dominer le travail.

Résultat / données d’exemple

order_id	full_name
101	Alice Martin
102	Bilal Amrani
103	Alice Martin

Distinguer la justesse de la condition de jointure de l’efficacité de sa procédure physique.

ANALYZE actualise les prévisions ; VACUUM récupère l’espace des anciennes lignes

Après de nombreux changements, les statistiques peuvent décrire les données d’hier. La commande autonome ANALYZE échantillonne le contenu actuel et actualise les statistiques du planificateur ; elle diffère de l’option de mesure dans EXPLAIN. Un tuple mort est une ancienne version de ligne dont aucun instantané actif ne peut encore avoir besoin. VACUUM rend réutilisable l’espace des anciennes versions supprimables et entretient les informations de visibilité. Autovacuum est le service de maintenance de fond de PostgreSQL. Ces commandes contribuent à la bonne santé d’une table, mais aucune ne garantit un plan plus rapide pour toute requête.

SQL / lire l’instruction une clause à la fois

```sql
ANALYZE work.bench_orders;

VACUUM (ANALYZE) work.bench_orders;
```

Sens des mots-clés

Commande ANALYZE — Recueillir de nouveaux résumés statistiques de la table.

Tuple mort — Ancienne version de ligne dont aucun instantané actif ne peut avoir besoin.

VACUUM — Rendre réutilisable le stockage des anciennes versions supprimables et entretenir les informations de visibilité.

Autovacuum — Service de fond planifiant le nettoyage et l’analyse.

La maintenance modifie les informations et l’espace réutilisable

Ce sont des observations illustratives, pas des sorties garanties par les commandes. Après analyse, une estimation des commandes pending peut se rapprocher des 30,000 lignes réelles sans changer cette réponse. Après nettoyage, l’espace des anciennes versions supprimables peut devenir réutilisable et davantage de pages peuvent être entièrement visibles. Les lignes logiques actuelles restent identiques. Si une ancienne transaction a encore besoin d’une ancienne version, le nettoyage doit la conserver ; une commande de maintenance réussie ne prouve pas que toutes les anciennes versions étaient supprimables.

Résultat / données d’exemple

Observation illustrative	Avant	Après
Estimation des lignes pending / réalité	3,000 / 30,000	29,800 / 30,000
Espace des anciennes versions réutilisable	Pas encore récupéré	Disponible pour de futures lignes
Lignes actuelles correspondantes	30,000	30,000

Le tri nécessite de la mémoire de travail et peut déborder vers des fichiers temporaires

Un tri réorganise les lignes dans l’ordre demandé. Ses données de travail doivent tenir quelque part : lorsque la mémoire de travail disponible est insuffisante, PostgreSQL peut déporter une partie du travail dans des fichiers temporaires, ajoutant des transferts de stockage. work_mem règle un tri individuel ou une opération similaire, pas une enveloppe mémoire unique pour tout le serveur. Une requête peut comporter plusieurs opérations de ce type et de nombreuses sessions peuvent s’exécuter ensemble ; augmenter cette valeur multiplie donc la demande mémoire possible.

SQL / lire l’instruction une clause à la fois

```sql
BEGIN;
SET LOCAL work_mem = '64kB';
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, total
FROM work.bench_orders
ORDER BY total, order_id;
ROLLBACK;
```

Sens des mots-clés

work_mem — Enveloppe mémoire de base par tri ou opération comparable.

Spill — Écrire les données de travail en fichiers temporaires si la mémoire ne suffit pas.

SET LOCAL — Appliquer un réglage uniquement jusqu’à la fin de cette transaction.

Un tri sur disque renseigne sur le travail effectué

Un plan réel peut indiquer Sort Method: external merge et une quantité Disk lorsque le tri déborde. Ces détails et quantités sont ici illustratifs. Les premières lignes exactes dans l’ordre demandé sont 10000, 20000 et 30000, chacune avec total 0.00. Trier les 300,000 lignes pour afficher toutes les commandes est une charge différente de l’affichage d’une petite page. Avant d’augmenter la mémoire, demandez-vous si l’application a besoin de toutes les lignes, de moins de colonnes ou d’un ordre pris en charge par un index utile.

Détails de plan illustratifs

Détail	Valeur d’exemple
Sort Key	total, order_id
Sort Method	external merge
Disk	7,000 kB

Un tri sur disque renseigne sur le travail effectué / résultat commenté

Un plan réel peut indiquer Sort Method: external merge et une quantité Disk lorsque le tri déborde. Ces détails et quantités sont ici illustratifs. Les premières lignes exactes dans l’ordre demandé sont 10000, 20000 et 30000, chacune avec total 0.00. Trier les 300,000 lignes pour afficher toutes les commandes est une charge différente de l’affichage d’une petite page. Avant d’augmenter la mémoire, demandez-vous si l’application a besoin de toutes les lignes, de moins de colonnes ou d’un ordre pris en charge par un index utile.

Trois premières lignes exactes du résultat trié

order_id	total
10000	0.00
20000	0.00
30000	0.00

À vous de jouer : améliorer la question avant d’augmenter les ressources

Un export quotidien utilise WHERE CAST(ordered_at AS date) = DATE '2025-01-02' et trie chaque ligne complète correspondante. Réécrivez le filtre sous forme de plage UTC semi-ouverte explicite. Donnez le nombre de lignes attendu sur ce banc d’essai, puis citez deux éléments à recueillir avant de recommander davantage de work_mem. Indice : la table contient une commande par minute et un nouveau réglage se justifie par le travail observé plutôt que par un souhait général d’avoir plus de mémoire.

Suivez le raisonnement

1. Gardez ordered_at inchangé à gauche des deux comparaisons.
2. Incluez minuit le 2 janvier et excluez minuit le 3 janvier.
3. Reliez la proposition de ressources au comportement mesuré du tri et à la concurrence.

Réponse : préserver le jour et recueillir des éléments sur le tri

Le filtre semi-ouvert inclut tous les instants du 2 janvier UTC et aucun du 3 janvier. Il correspond exactement à 1,440 lignes du banc d’essai. Avant de modifier work_mem, examinez la méthode de tri réelle et l’activité des fichiers temporaires, puis considérez la largeur des lignes et le nombre d’opérations susceptibles de s’exécuter en parallèle. Un tri tenant déjà en mémoire peut peu y gagner. Renvoyer seulement les colonnes nécessaires ou exploiter un ordre existant utile peut réduire le travail tout en préservant la sortie requise.

SQL / lire l’instruction une clause à la fois

```
SELECT count(*) AS day_orders
FROM work.bench_orders
WHERE ordered_at >= TIMESTAMPTZ '2025-01-02 00:00+00'
  AND ordered_at < TIMESTAMPTZ '2025-01-03 00:00+00';
```

Sens des mots-clés

>= borne inférieure — Inclure le début du jour UTC souhaité.

< borne supérieure — Exclure le début du jour suivant.

Corriger d’abord la sémantique, puis mesurer le travail restant.

OFFSET écarte les lignes précédentes avant de renvoyer une page

La pagination consiste à renvoyer un grand résultat ordonné en petites pages. OFFSET saute un nombre précisé de lignes avant que LIMIT compte les lignes renvoyées. Cette demande affiche trois identifiants après les 100,000 premiers. Même avec un index d’ordre, PostgreSQL doit calculer les lignes sautées ; une page lointaine peut donc demander un travail important. ORDER BY order_id donne un ordre unique, car order_id est la clé primaire ; cette page a donc un sens défini pour un instantané fixe.

SQL / lire l’instruction une clause à la fois

```sql
SELECT order_id, total
FROM work.bench_orders
ORDER BY order_id
LIMIT 3 OFFSET 100000;
```

Sens des mots-clés

Pagination — Renvoyer un grand résultat en petites pages ordonnées.

OFFSET 100000 — Sauter les 100,000 premières lignes du résultat.

LIMIT 3 — Renvoyer jusqu’à trois lignes après le saut.

La pagination par clé reprend après la dernière clé vue

La pagination par clé mémorise la dernière valeur d’ordre renvoyée et demande les valeurs suivantes. Si la page précédente se terminait à order_id 100000, la condition order_id > 100000 décrit directement la suite. Le B-tree existant de clé primaire peut atteindre cette borne au lieu de calculer un long préfixe ignoré. Cela convient à une interface de page suivante plutôt qu’aux sauts arbitraires vers des pages numérotées. La clé d’ordre doit être unique ou inclure un critère unique de départage pour que des valeurs égales n’entraînent pas de lignes manquantes.

SQL / lire l’instruction une clause à la fois

```sql
SELECT order_id, total
FROM work.bench_orders
WHERE order_id > 100000
ORDER BY order_id
LIMIT 3;
```

Sens des mots-clés

Pagination par clé — Reprendre à partir de valeurs d’ordre mémorisées plutôt que d’un nombre de lignes.

> 100000 — Commencer strictement après la dernière clé déjà affichée.

Critère de départage — Clé supplémentaire rendant uniques des valeurs d’ordre égales.

Les deux demandes de page renvoient les mêmes lignes sur les données fixes

Sur le banc d’essai inchangé, les deux requêtes de page renvoient les identifiants 100001, 100002 et 100003 avec les totaux 1.00, 2.00 et 3.00. OFFSET exprime combien de lignes précédentes écarter ; la pagination par clé exprime où commencer la recherche. Leur concordance ici dépend de la correspondance entre la clé mémorisée et la même borne sur des données fixes. C’est une comparaison commentée de la sémantique et du travail d’accès potentiel, pas un rapport de vitesse mesuré ni une garantie pour tout plan possible.

Résultat / données d’exemple

order_id	total	Renvoyé par
100001	1.00	OFFSET et pagination par clé
100002	2.00	OFFSET et pagination par clé
100003	3.00	OFFSET et pagination par clé

Une borne de reprise peut éviter de recalculer un préfixe écarté.

Les partitions de stockage divisent une table en éléments physiques

Le partitionnement de table divise une table logique en tables filles physiques plus petites selon une clé, comme le mois. Il diffère du PARTITION BY d’une fonction de fenêtrage, qui regroupe des lignes pour un calcul sans modifier le stockage. L’élagage des partitions consiste à exclure les tables filles dont les bornes ne peuvent contenir de correspondances. Les partitions de stockage mensuelles peuvent aussi servir un cycle de vie des données : garder les mois récents, archiver les anciens et supprimer volontairement les éléments expirés. Le partitionnement ajoute du travail de conception et de maintenance et n’améliore pas automatiquement la vitesse.

Résultat / données d’exemple

Question logique	Table fille de janvier	Table fille de février	Table fille de mars
Tous les mois	Lire si nécessaire	Lire si nécessaire	Lire si nécessaire
Plage de dates de février	Élaguer	Candidate à la lecture	Élaguer
Politique de conservation	Archiver après approbation	Conserver	Conserver

Créer un petit exemple jetable partitionné par plage / contexte et définitions

PARTITION BY RANGE déclare comment le parent oriente les lignes. PARTITION OF crée une table fille physique appartenant à ce parent. FOR VALUES FROM et TO définissent une borne inférieure inclusive et une borne supérieure exclusive, correspondant à nos plages de dates semi-ouvertes. Ce petit exemple indépendant stocke des identifiants et dates de commandes de janvier et février. INSERT vise le parent ; PostgreSQL oriente chaque ligne vers la table fille dont les bornes contiennent sa date. La table du banc d’essai reste inchangée.

Sens des mots-clés

PARTITION BY RANGE — Orienter les lignes de la table parent selon les plages de cette clé.

PARTITION OF — Créer une table fille appartenant au parent nommé.

FOR VALUES FROM ... TO — Définir des bornes inférieure inclusive et supérieure exclusive.

Créer un petit exemple jetable partitionné par plage

Stocker un historique jetable de deux mois de commandes dans des éléments physiques par plage de dates afin que les requêtes mensuelles puissent élaguer les autres.

SQL / lire l’instruction une clause à la fois

```
CREATE TABLE work.order_months (
  order_id integer, ordered_on date
) PARTITION BY RANGE (ordered_on);
CREATE TABLE work.order_jan
PARTITION OF work.order_months
FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');
CREATE TABLE work.order_feb
PARTITION OF work.order_months
FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');
INSERT INTO work.order_months VALUES
  (101, '2026-01-05'), (104, '2026-02-01');
```

Une condition de février rend janvier impossible

Pour cette requête complète, les bornes de janvier ne peuvent contenir de ligne admissible ; l’élagage des partitions peut donc exclure cette table fille. Le résultat exact est la commande 104 datée du 2026-02-01. Ajoutez EXPLAIN avant ce SELECT pour examiner quelle table fille apparaît : un parcours de order_feb peut produire cette réponse sans parcourir order_jan. L’élagage repose sur les bornes des partitions et les conditions, pas sur l’existence d’un index. Une condition mal adaptée ou une question nécessitant tous les mois peut encore demander de nombreuses tables filles.

SQL / lire l’instruction une clause à la fois

```sql
SELECT order_id, ordered_on
FROM work.order_months
WHERE ordered_on >= DATE '2026-02-01'
  AND ordered_on < DATE '2026-03-01'
ORDER BY order_id;
```

Sens des mots-clés

>= borne inférieure — Inclure le 1er février.

< borne supérieure — Exclure le 1er mars et les dates suivantes.

Une requête préparée accepte une valeur à l’exécution

Un paramètre est une valeur fournie séparément lors de l’exécution d’une requête. PREPARE nomme une instruction dans la session actuelle et $1 désigne son premier paramètre. EXECUTE fournit cette valeur ; DEALLOCATE supprime l’instruction préparée. Un plan personnalisé est élaboré à partir d’une valeur fournie particulière, tandis qu’un plan générique est réutilisable pour différentes valeurs. PostgreSQL choisit automatiquement entre eux. La préparation peut économiser du travail de planification répété, mais ne garantit pas que chaque valeur de paramètre bénéficie du même parcours physique.

SQL / lire l’instruction une clause à la fois

```sql
PREPARE queue_count(text) AS
SELECT count(*) AS orders
FROM work.bench_orders
WHERE status = $1;

EXECUTE queue_count('pending');
EXPLAIN EXECUTE queue_count('paid');
DEALLOCATE queue_count;
```

Une requête préparée accepte une valeur à l’exécution / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

PREPARE ... AS — Nommer une instruction pour la réutiliser dans cette session.

$1 — Emplacement du premier paramètre fourni.

EXECUTE — Exécuter l’instruction préparée avec les valeurs de paramètres fournies.

DEALLOCATE — Supprimer l’instruction préparée nommée.

Plan personnalisé / générique — Plan propre à une valeur / plan réutilisable pour différentes valeurs.

Des paramètres différents peuvent décrire des quantités de travail différentes

Exécuter queue_count('pending') renvoie le nombre exact 30,000. La condition paid compterait 270,000 lignes, bien que la forme SQL soit identique. Un plan personnalisé peut tenir compte de la fréquence de la valeur fournie ; un plan générique doit convenir sans cette valeur particulière. Dans EXPLAIN EXECUTE, un plan générique peut afficher $1, tandis qu’un plan personnalisé montre la valeur fournie. C’est une raison d’examiner des paramètres représentatifs lors de la mesure d’une requête applicative, pas de forcer immédiatement un type de plan.

Sortie exacte de EXECUTE queue_count('pending')

orders
30000

Comparaison de paramètres issue des données du banc d’essai

Statut fourni	Lignes correspondantes
pending	30,000
paid	270,000

À vous de jouer : rédiger une brève recommandation étayée

CampusMarket veut un écran de commandes à page suivante, une petite file pending et une conservation mensuelle des données. Recommandez une technique de départ adaptée à chaque besoin avec le vocabulaire du jour, puis décrivez une mesure équitable pour l'écran. Incluez une limite ou une vérification pour chaque proposition. Indice : adaptez la technique à la question : reprise après une clé mémorisée, condition définissant un sous-ensemble ou éléments physiques régis par des bornes de dates. Évitez de présenter ces techniques comme des remèdes universels à la lenteur de SQL.

Suivez le raisonnement

1. Associez l'écran à page suivante, la file pending et le besoin de conservation à une technique concrète.
2. Indiquez une vérification ou une limite susceptible de modifier chaque décision.
3. Décrivez des mesures répétées comparables et la vérification de la justesse du résultat.

Réponse : relier chaque proposition à une question et une vérification

Utilisez la pagination par clé pour reprendre après la dernière clé de commande unique, un index partiel comme candidat pour une file limitée à pending, et des partitions de stockage mensuelles lorsque la conservation et la charge justifient la gestion supplémentaire. Vérifiez respectivement la sémantique de la borne, l’admissibilité du prédicat et l’élagage par bornes de dates. Comparez des résultats identiques avec des exécutions répétées à chaud comparables et leur médiane, puis examinez les lignes réelles et les tampons. Un bon travail de performance explique pourquoi un changement devrait réduire le travail observé et vérifie cette attente sans promettre d’accélération fixe.

Résultat / données d’exemple

Besoin	Technique de départ	Vérification requise
Passer aux commandes suivantes	Pagination par clé	Ordre unique stable et borne mémorisée correcte.
Lire la file pending	Index partiel candidat	La requête prouve le prédicat pending ; mesurer le coût d’écriture.
Gérer la conservation mensuelle	Partitions de stockage	Bornes, élagage et cycle de conservation approuvé.

Poser la question, examiner le plan, changer un élément et vérifier les résultats corrects ainsi que le travail mesuré.